

DOSSIER DE CONCEPTION

AMBAWBIO SUITE

« Celui qui connaît demain »

Étude de l'architecture

Architecture technique client-serveur

Suite de gestion d'entreprise intégrée « All-in-One », conçue pour le Burkina Faso

Élément	Valeur
Référence	AMB-CONC-04
Version	1.1
Date	Octobre 2026
Porteur du projet	Rachid Bonsa
Statut	Version validé
Diffusion	Confidentiel

Sommaire

1. Introduction
2. Principes et contraintes d'architecture
3. Vue d'ensemble
4. Choix technologiques
5. Architecture du serveur
6. Architecture du client
7. Architecture des données
8. Moteur hors-ligne et synchronisation
9. Intégrations externes
10. Sécurité
11. Déploiement et exploitation
12. Traçabilité des exigences
13. Registre des décisions d'architecture
14. Décisions validées

1. Introduction

1.1 Objet du document

Ce document est le quatrième livrable du dossier de conception d'Ambawbio Suite. Il décrit l'architecture technique retenue : les composants, les technologies, l'organisation du code, le modèle de données, le moteur hors-ligne, les intégrations, la sécurité et le déploiement. Il sert de référence à l'équipe de développement et prépare les diagrammes UML (cas d'utilisation, classes, séquences).

1.2 Documents d'entrée

Livrable	Éléments repris
AMB-CONC-01, Parties prenantes	Exigences EX-01 à EX-14 (FEC, hors-ligne, Mobile Money, données personnelles, hébergement local).
AMB-CONC-02, Étude fonctionnelle et MoSCoW	156 fonctionnalités, 18 modules, exigences non fonctionnelles ENF-01 à ENF-16, règles de gestion RG-01 à RG-14, découpage en releases.
AMB-CONC-03, Analyse des risques	Choix de conception imposés par les risques (section 9) : module de conformité isolé, synchronisation idempotente, isolation multi-tenant, ports de paiement.

1.3 Choix structurants du porteur du projet

- **Architecture client-serveur**, sans microservices : un seul serveur d'application, découpé en modules internes (monolithe modulaire).
- **Back-end** : Java et Spring Boot.
- **Front-end** : Angular et Tailwind CSS.
- **Base de données** : PostgreSQL.

2. Principes et contraintes d'architecture

ID	Principe	Justification
PA-01	Monolithe modulaire : une seule application déployée, des modules aux frontières strictes.	Choix du porteur ; simplicité de déploiement et d'exploitation pour une petite équipe ; transactions locales fiables pour la comptabilité.
PA-02	Hors-ligne d'abord pour les opérations de terrain.	ENF-01, R07, R08 : coupures d'électricité et de réseau.
PA-03	Une seule base de données, un seul référentiel.	Promesse « une seule vérité » de la proposition ; pas de ressaisie.
PA-04	Conformité isolée et paramétrable.	R01, R03 : spécifications FEC et barèmes évolutifs.
PA-05	Intégrations derrière des ports (interfaces) interchangeables.	R15, R16 : simulateur de paiement d'abord, agrégateur ensuite, opérateurs directs plus tard.
PA-06	Sécurité et isolation multi-tenant dès la conception.	R12, R13, ENF-05, ENF-06.
PA-07	Sobriété réseau et matérielle.	ENF-02, ENF-03, ENF-04 : réseau 3G, terminaux modestes.
PA-08	Souveraineté et réversibilité.	EX-09 : hébergement au Burkina Faso ou sur site, export complet.
PA-09	Technologies standard, documentées et répandues.	R24 : faciliter le recrutement et la formation des développeurs.

2.1 Pourquoi un monolithe modulaire plutôt que des microservices

Critère	Monolithe modulaire (retenu)	Microservices
Exploitation	Une application, une base : simple à héberger au Burkina Faso et à installer sur site.	Nombreux services, orchestrateur, courtier de messages : coûteux et complexe.
Cohérence des données	Transactions locales : une vente, son stock et son écriture comptable sont cohérents.	Cohérence à terme, transactions distribuées.
Taille de l'équipe	Adapté à une petite équipe.	Adapté à de nombreuses équipes autonomes.
Installation sur site	Un seul serveur suffit.	Infrastructure lourde chez le client.
Évolutivité	Mise à l'échelle horizontale (plusieurs instances identiques) ; extraction d'un module plus tard si nécessaire.	Mise à l'échelle fine par service.

3. Vue d'ensemble

Ambawbio Suite suit une architecture client-serveur en trois niveaux : les clients (application web, application Android, portail et boutique), le serveur d'application unique, et la couche de données. Le serveur communique avec les services externes (Mobile Money, DGI, messagerie, banques).

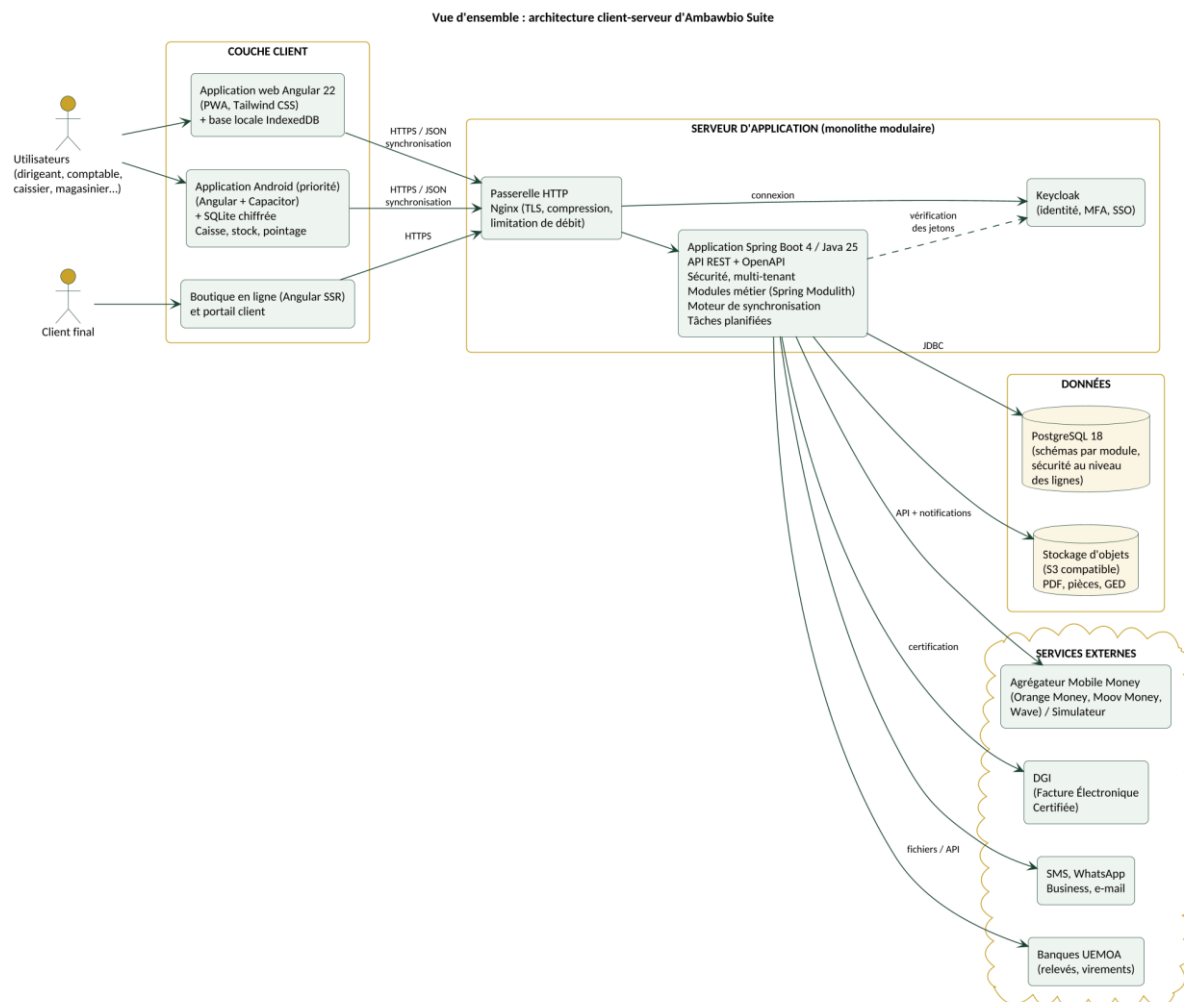


Figure 1 : Vue d'ensemble de l'architecture

Niveau	Composant	Rôle
Client	Application web Angular (PWA)	Interface principale pour tous les profils, installable, avec cache local pour le hors-ligne.
Client	Application Android (Angular + Capacitor)	Même code Angular, emballé pour Android ; base SQLite chiffrée et persistante pour la caisse, le stock et le pointage.
Client	Portail client et boutique en ligne	Boutique et pages publiques rendues côté serveur (Angular SSR) pour le référencement et les faibles débits ; portail client (espace connecté) rendu dans le navigateur.
Serveur	Passerelle Nginx	Terminaison TLS, compression, limitation de débit, fichiers statiques.
Serveur	Application Spring Boot	API REST, sécurité, modules métier, synchronisation, tâches planifiées, intégrations.
Données	PostgreSQL	Données métier, journal d'audit, flux de synchronisation, événements.
Données	Stockage d'objets	Factures PDF, pièces justificatives, documents de la GED.

4. Choix technologiques

Les versions indiquées sont les versions stables disponibles en octobre 2026. L'équipe adoptera la dernière version stable au démarrage du développement, puis suivra les versions à support long.

4.1 Serveur

Besoin	Technologie	Justification
Langage	Java 25 (version à support long)	Langage préféré du porteur, robuste, très répandu.
Cadre applicatif	Spring Boot 4.x (Spring Framework 7)	Standard du marché, écosystème complet.
Modularité	Spring Modulith 2.x	Vérifie les frontières des modules, gère les événements entre modules avec enregistrement en base, documente l'architecture.
API	Spring Web (REST, JSON) + springdoc-openapi	API documentée automatiquement (OpenAPI), utilisable par les partenaires.
Sécurité	Spring Security (serveur de ressources OAuth2) + Keycloak	Keycloak gère l'identité (connexion, MFA, politique de mots de passe, protection contre les attaques par force brute, SSO, annuaires LDAP/Active Directory des grands comptes) ; Spring Security vérifie les jetons et applique les droits métier.
Accès aux données	Spring Data JPA (Hibernate) + jOOQ ou JDBC pour les rapports	Productivité pour le métier, requêtes SQL optimisées pour la comptabilité et les rapports.
Migrations de schéma	Flyway	Évolutions de base versionnées et reproductibles, par module.
Correspondance objets	MapStruct	Conversion entités / objets de transfert sans code répétitif.
Tâches planifiées	Spring Scheduling + ShedLock	Relances, clôtures, synchronisations ; une seule exécution même avec plusieurs instances.
Documents	JasperReports (PDF), Apache POI (Excel)	Factures, bulletins, états financiers, exports.
Cache	Caffeine (en mémoire)	Cache local simple, sans serveur supplémentaire.
Tests	JUnit 5, Testcontainers, Spring Modulith Test, ArchUnit	Tests sur une vraie base PostgreSQL, vérification des frontières de modules.

4.2 Client

Besoin	Technologie	Justification
Cadre applicatif	Angular 22 (composants autonomes, signaux)	Choix du porteur ; structure adaptée aux grandes applications de gestion.
Style	Tailwind CSS 4	Choix du porteur ; feuilles de style légères, système de conception cohérent.
Hors-ligne (web)	Angular Service Worker + Dexie.js (IndexedDB)	Cache des écrans et ressources ; base locale structurée dans le navigateur.
Application Android	Capacitor + SQLite chiffrée	Réutilise le code Angular ; stockage persistant et chiffré, que le système ne peut pas effacer (R08).
Traductions	Transloco	Changement de langue sans recompilation ; français au lancement.
Graphiques	ECharts (ngx-echarts)	Tableaux de bord riches et légers.

Besoin	Technologie	Justification
Impression	Impression ESC/POS via Capacitor, PDF côté web	Tickets thermiques en caisse.
Tests	Vitest, Playwright	Tests unitaires et tests de bout en bout.

4.3 Données et infrastructure

Besoin	Technologie	Justification
Base de données	PostgreSQL 18	Choix du porteur ; transactions fiables, JSONB pour les champs personnalisés, sécurité au niveau des lignes, recherche plein texte.
Fichiers	Stockage d'objets compatible S3 (MinIO en auto-hébergement)	Installable chez un hébergeur local ou sur site.
Conteneurs	Docker, Docker Compose	Même livrable pour le SaaS et l'installation sur site.
Passerelle	Nginx	TLS, compression, répartition de charge.
Identité	Keycloak (base PostgreSQL dédiée)	Fournisseur d'identité standard (OpenID Connect) ; installable en SaaS comme sur site.
Sauvegardes	pgBackRest (sauvegardes + archivage WAL)	Restauration à un instant donné ; perte maximale d'une heure (ENF-13).
Supervision	Micrometer, Prometheus, Grafana, journaux centralisés	Suivi de la disponibilité (ENF-12) et des alertes.
Intégration continue	GitHub (code source) + GitHub Actions	Compilation, tests, analyse de code, construction des images.
Qualité du code	SonarQube, Checkstyle, ESLint	Contrôle continu de la qualité et de la sécurité.

5. Architecture du serveur

5.1 Modules

Le serveur est une seule application Spring Boot, divisée en modules qui correspondent aux domaines fonctionnels. Un module ne peut utiliser que l'API publique d'un autre module. Les flux importants entre modules (par exemple une vente qui doit produire une écriture comptable) passent par des **événements de domaine**. Spring Modulith enregistre ces événements dans PostgreSQL dans la même transaction que l'opération : aucun événement n'est perdu, même en cas d'arrêt du serveur, et aucun courtier de messages n'est nécessaire.

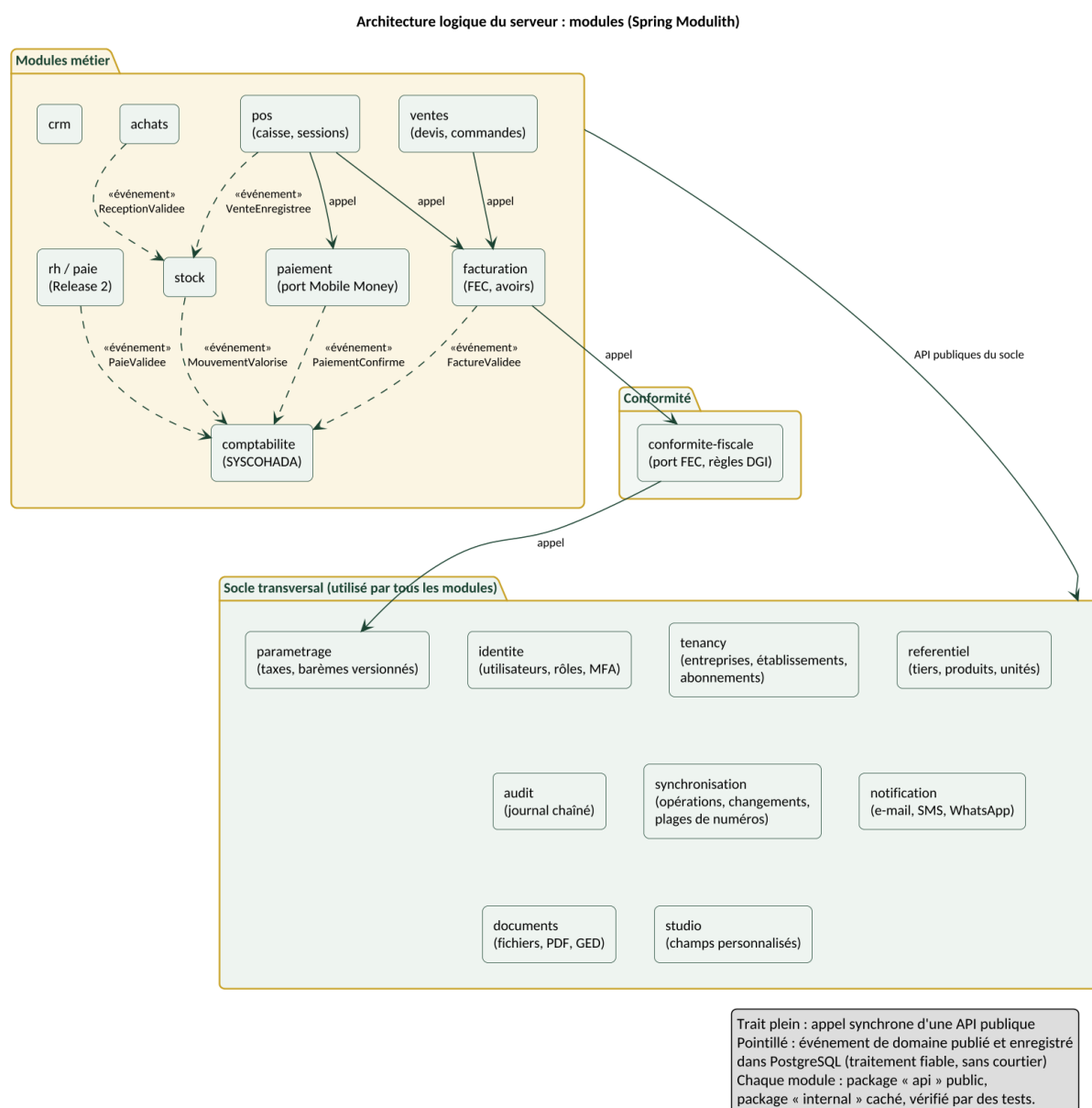


Figure 2 : Modules du serveur et leurs relations

5.2 Organisation du code

Chaque module suit la même structure en couches. Le package `api` contient ce que les autres modules peuvent utiliser ; le package `internal` est invisible de l'extérieur.

```

bf.ambawbio
├── AmbawbioApplication.java
├── socle/          identite, tenancy, referentiel, parametrage, audit,
                   synchronisation, notification, documents, studio
├── conforme/      conforme-fiscale (port FEC, règles DGI)
├── ventes/
│   ├── api/       VentesApi, événements publics (DevisAccepte...), DTO
│   └── internal/
│       ├── web/    contrôleurs REST
│       ├── application/ services, cas d'utilisation, transactions
│       ├── domain/ entités, règles de gestion, événements
│       └── infrastructure/ dépôts JPA, adaptateurs
├── pos/ facturation/ paiement/ comptabilite/
├── stock/ achats/ crm/ rh/ paie/ ...
├── shared/         types communs : Montant (FCFA), Ifu, identifiants

```

5.3 Conventions de l'API REST

- Ressources versionnées : `/api/v1/{module}/{ressource}` ; format JSON ; documentation OpenAPI générée.
- Le contexte de l'entreprise et de l'établissement est tiré du jeton d'accès, jamais d'un paramètre libre.
- Pagination, tri et filtres standardisés ; réponses compressées (gzip ou brotli).
- Écritures idempotentes : chaque création porte un identifiant généré par le client (UUID v7), ce qui permet de rejouer une requête sans créer de doublon.
- Erreurs au format standard « Problem Details » (RFC 9457), avec des messages en français.

7. Architecture des données

7.1 Organisation de la base

- **Une base PostgreSQL**, avec **un schéma par module** (socle, ventes, pos, facturation, comptabilité, stock...). Chaque module ne lit et n'écrit que dans son schéma ; il accède aux données des autres modules par leur API.
- **Migrations Flyway par module**, versionnées avec le code.
- **Identifiants UUID v7**, générés par le client ou par le serveur. Ils sont triables dans le temps et permettent de créer des enregistrements hors-ligne sans conflit d'identifiant.
- **Montants** stockés en entiers (francs CFA sans décimales) ou en décimal exact pour les devises étrangères ; jamais en nombres à virgule flottante.
- **Champs personnalisés** du Studio stockés dans une colonne JSONB indexée sur chaque table extensible.
- **Horodatage** de création et de modification, version de ligne pour détecter les modifications concurrentes.

7.2 Multi-tenant

En mode SaaS, toutes les entreprises partagent la même base. Chaque table métier porte une colonne `tenant_id`. L'isolation est garantie à deux niveaux :

1. **Niveau applicatif** : un filtre Hibernate ajoute automatiquement la condition sur l'entreprise à chaque requête.
2. **Niveau base de données** : la sécurité au niveau des lignes de PostgreSQL (Row-Level Security) refuse toute ligne d'une autre entreprise, même en cas de défaut dans le code. L'application positionne l'entreprise courante dans la session de base de données au début de chaque transaction.

En installation sur site, une instance est dédiée au client : le même code fonctionne avec une seule entreprise.

7.3 Tables techniques transverses

Table	Rôle
audit.journal	Journal d'audit : chaque ligne contient l'empreinte de la ligne précédente (chaînage SHA-256), ce qui rend toute altération détectable.
sync.operation_recue	Opérations reçues des terminaux, avec leur identifiant unique (idempotence).
sync.flux_changements	Changements numérotés par une séquence serveur, lus par les terminaux à partir de leur curseur.
sync.terminal	Terminaux enregistrés (caisse, téléphone), leur clé et leur dernière synchronisation.
sync.plage_numerotation	Plages de numéros de pièces réservées à chaque terminal.
event_publication	Événements entre modules (Spring Modulith), rejoués en cas d'échec.
parametrage.bareme	Barèmes fiscaux et sociaux versionnés par date d'effet (RG-10).

7.4 Conservation et sauvegardes

- Sauvegarde complète quotidienne et archivage continu des journaux de transactions (WAL) : restauration à un instant donné, perte maximale d'une heure.

- Règle 3-2-1 : trois copies, deux supports, une copie hors site (site de secours géographiquement séparé), toutes chiffrées.
- Factures, écritures et journal d'audit conservés pendant la durée légale ; partitionnement par année pour garder la base performante.
- Test de restauration trimestriel.

8. Moteur hors-ligne et synchronisation

Le moteur de synchronisation est la pierre angulaire d'Ambawbio Suite. Il est développé en premier (risque R07).

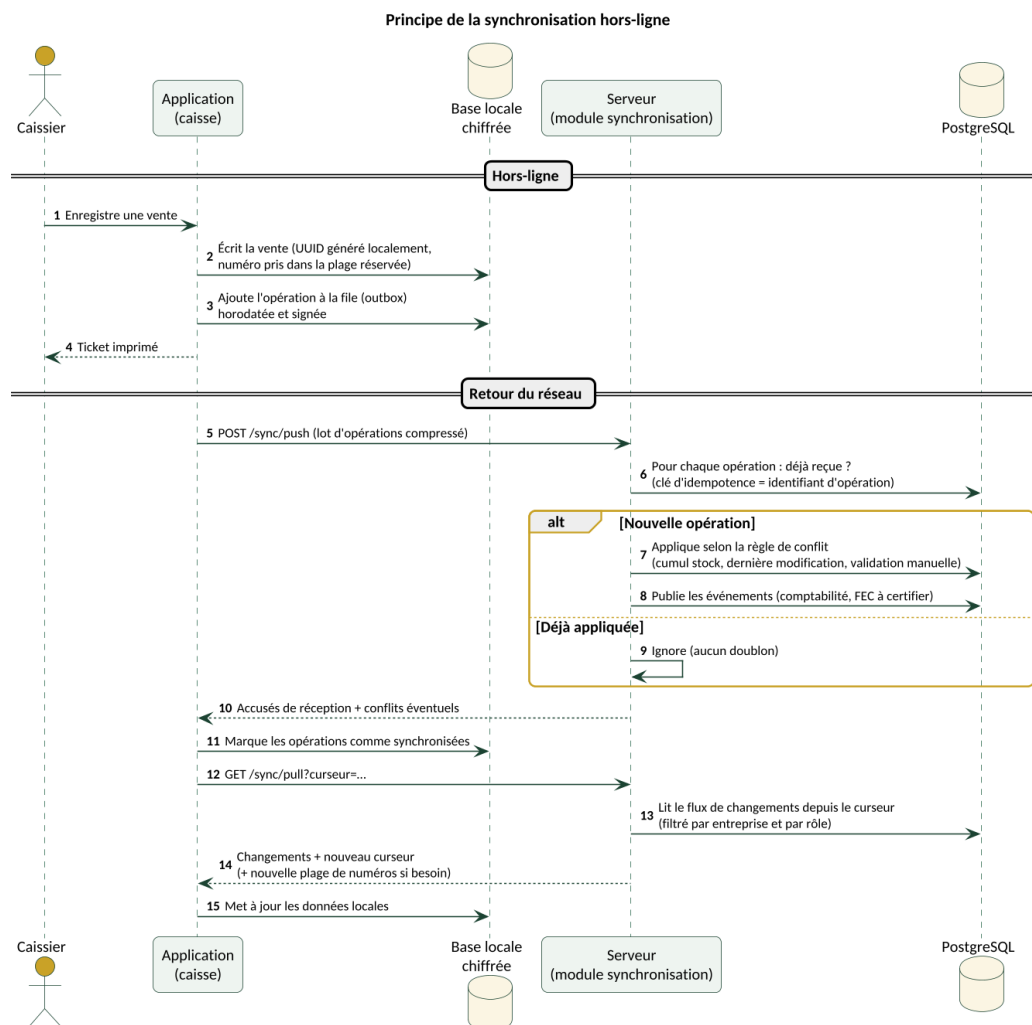


Figure 4 : Principe de la synchronisation

8.1 Principes

Élément	Fonctionnement
Enregistrement du terminal	Chaque caisse ou téléphone est enregistré auprès du serveur et reçoit un identifiant et une clé.
Données locales	Le terminal ne télécharge que les données utiles à son rôle et à son établissement (catalogue, prix, clients, stock du dépôt).
Opérations	Chaque action hors-ligne est une opération horodatée, signée et identifiée de façon unique. Elle est stockée dans une file locale.
Envoi (push)	Au retour du réseau, les opérations partent par lots compressés, dans l'ordre, avec reprise après coupure.
Idempotence	Le serveur ignore une opération déjà reçue : une coupure pendant l'envoi ne crée jamais de doublon.
Réception (pull)	Le terminal demande les changements survenus depuis son dernier curseur.

Élément	Fonctionnement
Conflits	Mouvements de stock cumulés (RG-05) ; fiches : dernière modification retenue (RG-06) ; prix et taxes modifiables uniquement en ligne ; cas sensibles soumis à validation manuelle.
Numérotation	Plages de numéros réservées par terminal ; alerte à 80 % (RG-03, RG-04).
Priorités	Ventes et encaissements d'abord, puis stock, puis le reste.
Surveillance	Alerte visible sur le terminal si des opérations ne sont pas synchronisées depuis plus de 24 heures ; tableau de bord côté serveur.

8.2 Cas de la facture certifiée

Une vente hors-ligne produit un ticket et une facture numérotée dans la plage du terminal. La facture est placée dans une file de certification et transmise à la DGI dès que la connexion revient. Le traitement exact (mention « en attente de certification », délai maximal) sera aligné sur les spécifications officielles de la FEC (risque R02).

9. Intégrations externes

Chaque service externe est caché derrière un **port** (une interface Java) défini par le métier. Les **adaptateurs** implémentent ce port pour un fournisseur donné. Le module métier ne sait pas s'il parle au simulateur, à l'agrégateur ou à un opérateur.

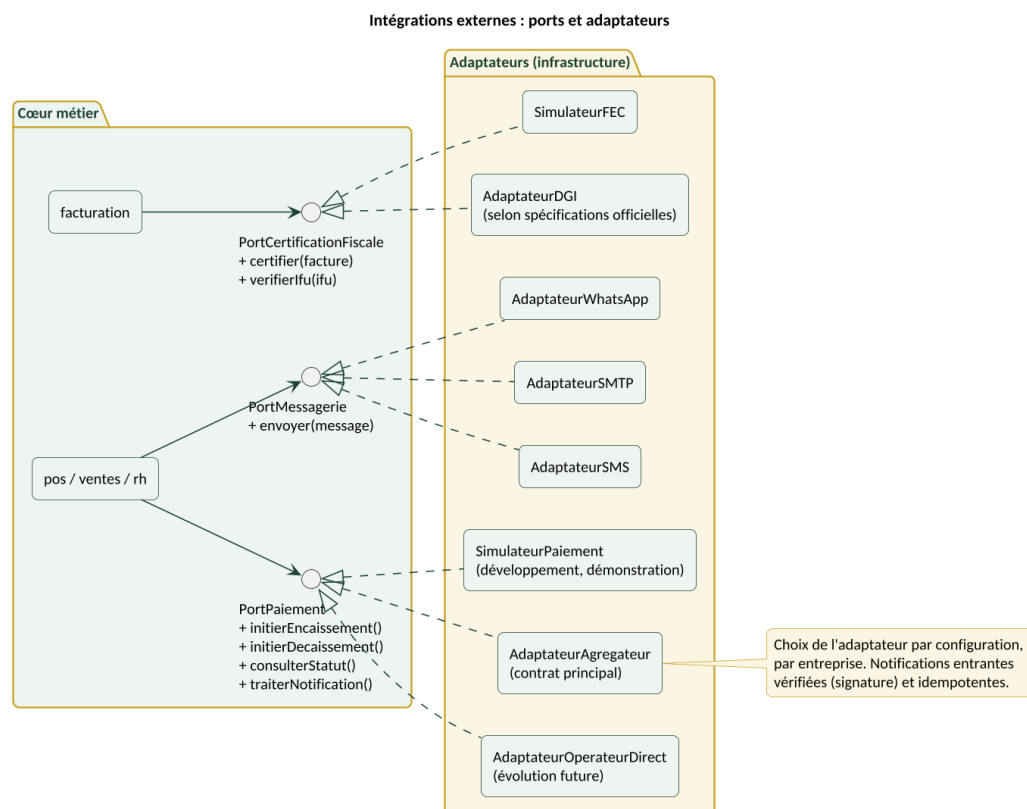


Figure 5 : Ports et adaptateurs des intégrations

Port	Adaptateurs	Points d'attention
PortPaiement	SimulateurPaiement (MVP), AdaptateurAgregateur (au plus tard en V1.0), AdaptateurOperateurDirect (futur)	Notifications entrantes vérifiées (signature) et idempotentes ; statut « en attente » tant que la notification n'est pas reçue (RG-07) ; rapprochement automatique.
PortCertificationFiscale	SimulateurFEC, AdaptateurDGI	File de certification avec réessais ; conservation de la réponse de la DGI (identifiant, QR code) ; mise à jour selon les spécifications officielles.
PortMessagerie	AdaptateurSMS, AdaptateurWhatsApp, AdaptateurSMTP	Envois asynchrones, suivi des statuts, respect du consentement.
PortBanque	Import de relevés (fichiers), virements groupés	Formats des banques de l'UEMOA ; connecteurs par partenariat.

Le simulateur de paiement reproduit les comportements réels : succès, refus, délai, notification en retard, notification en double. Il permet de développer, de tester et de faire des démonstrations sans compte Mobile Money.

10. Sécurité

Domaine	Mesures
Authentification	Keycloak : mots de passe hachés, politique de complexité, MFA par code à usage unique (obligatoire pour les rôles sensibles), verrouillage après échecs, SSO et connexion aux annuaires LDAP / Active Directory des grandes organisations. Jetons d'accès de courte durée (15 minutes) et jetons de renouvellement à rotation. Hors-ligne, l'application Android déverrouille la session locale par code PIN du caissier.
Autorisation	Rôles et permissions par module et par action, limités aux établissements autorisés ; contrôle côté serveur systématique.
Isolation	Double barrière : filtre applicatif et sécurité au niveau des lignes de PostgreSQL ; tests automatisés d'isolation.
Chiffrement	TLS 1.2 minimum (1.3 de préférence) ; chiffrement des disques et des sauvegardes ; chiffrement des champs sensibles ; base locale des terminaux chiffrée.
Traçabilité	Journal d'audit chaîné ; journaux techniques centralisés sans données personnelles en clair.
Protection applicative	Référentiel OWASP ASVS niveau 2 ; protection contre les injections, XSS, CSRF ; en-têtes de sécurité ; limitation de débit ; analyse des dépendances.
Secrets	Clés et mots de passe hors du code, dans des variables d'environnement ou un coffre de secrets ; rotation régulière.
Données personnelles	Registre des traitements, consentement, droits d'accès et d'effacement (anonymisation, RG-12), hébergement au Burkina Faso.
Vérification	Tests d'intrusion avant le lancement commercial, puis chaque année.

11. Déploiement et exploitation

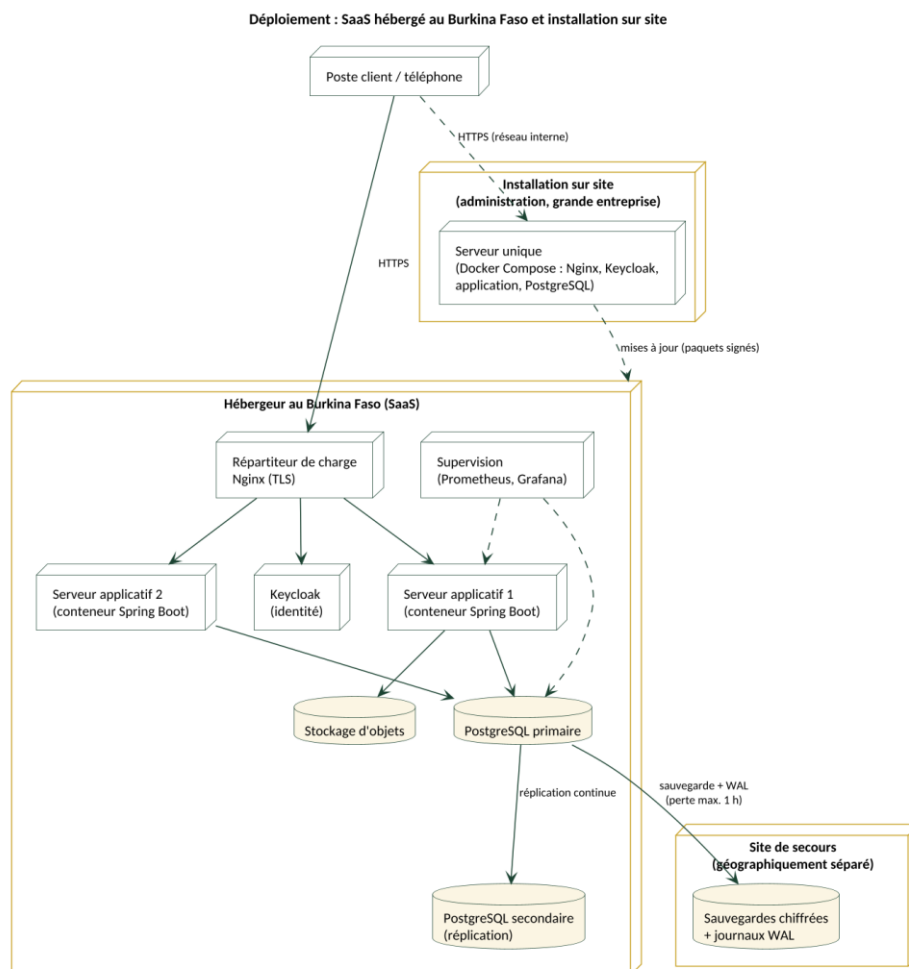


Figure 6 : Architecture de déploiement

11.1 Modes de déploiement

Mode	Architecture	Cible
SaaS	Chez un hébergeur au Burkina Faso : répartiteur Nginx, Keycloak, au moins deux instances de l'application (sans état, interchangeables), PostgreSQL primaire et secondaire, stockage d'objets, supervision ; sauvegardes vers un site de secours.	PME, commerces, associations
Sur site	Un serveur chez le client, installé avec Docker Compose (Nginx, Keycloak, application, PostgreSQL, stockage) ; mises à jour par paquets signés.	Administrations, banques, grandes entreprises
Hybride	Données sensibles sur site ; modules collaboratifs et boutique en ligne dans le SaaS, synchronisés de façon sécurisée.	Groupes multi-sites, entreprises industrielles

11.2 Environnements et livraison

Environnement	Usage
Développement	Poste du développeur, base PostgreSQL en conteneur, simulateurs de paiement et de FEC.
Intégration	Déployé automatiquement à chaque fusion ; tests automatisés complets.
Recette	Validation fonctionnelle par le porteur et les experts métier.

Environnement	Usage
Préproduction	Copie anonymisée de la production ; tests de performance et de migration.
Production	Clients réels ; déploiement sans interruption, instance par instance.

Chaque modification passe par la chaîne d'intégration continue : compilation, tests unitaires et d'intégration, vérification des frontières de modules, analyse de qualité et de sécurité, construction de l'image Docker, déploiement automatique en intégration.

12. Traçabilité des exigences

Exigence	Réponse de l'architecture	Section
ENF-01 Hors-ligne 7 jours	Base locale persistante et chiffrée, file d'opérations, plages de numéros	6, 8
ENF-02 Faible débit	Compression, synchronisation différentielle, modules chargés à la demande, pages publiques rendues côté serveur	5.3, 6, 8
ENF-03 Performance	Opérations de caisse locales, cache, requêtes SQL optimisées	6, 4.1
ENF-04 Terminaux	Angular allégé, application Android, impression ESC/POS	4.2, 6
ENF-05 Sécurité	TLS, Argon2id, MFA, OWASP ASVS niveau 2, chiffrement	10
ENF-06 Isolation	Filtre applicatif + sécurité au niveau des lignes	7.2
ENF-07 Conformité	Module de conformité fiscale, barèmes versionnés, registre CIL	5, 7.3, 10
ENF-08 Conservation	Partitionnement annuel, journal chaîné, archivage	7.3, 7.4
ENF-10 Maintenabilité	Monolithe modulaire vérifié, tests, intégration continue	5, 11.2
ENF-11 Langue	Transloco, textes externalisés	4.2
ENF-12 Disponibilité	Deux instances, base répliquée, supervision	11
ENF-13 Continuité	pgBackRest, WAL, règle 3-2-1, site de secours	7.4, 11
ENF-14 Capacité	Application sans état, mise à l'échelle horizontale	11
ENF-15 Réversibilité	Export complet par entreprise en formats ouverts	7

13. Registre des décisions d'architecture

ID	Décision	Alternatives écartées	Raison
ADR-01	Monolithe modulaire avec Spring Modulith	Microservices ; monolithe sans modules	Choix du porteur, simplicité, frontières vérifiées
ADR-02	Événements internes enregistrés dans PostgreSQL	Kafka, RabbitMQ	Pas de composant supplémentaire à exploiter
ADR-03	Base partagée + sécurité au niveau des lignes	Une base ou un schéma par entreprise	Coût d'exploitation, migrations uniques
ADR-04	Identifiants UUID v7	Séquences numériques	Création hors-ligne sans conflit
ADR-05	Application Android Capacitor pour les usages critiques	PWA seule ; application native Kotlin	Persistance garantie, réutilisation du code Angular
ADR-06	Ports et adaptateurs pour paiement, FEC, messagerie	Appels directs aux API	Simulateurs, changement de fournisseur
ADR-07	API REST + OpenAPI	GraphQL	Simplicité, cache HTTP, outillage
ADR-08	Keycloak comme fournisseur d'identité, Spring Security comme serveur de ressources	Authentification développée sur mesure	Solution éprouvée et maintenue ; MFA, SSO et LDAP prêts à l'emploi ; moins de code de sécurité à écrire
ADR-10	Angular SSR pour la boutique et les pages publiques ; rendu navigateur pour le reste	Tout en rendu navigateur	Référencement, affichage rapide en 3G
ADR-11	Application Android développée en priorité	Web d'abord	Caisse et terrain : persistance des données et impression des tickets

ID	Décision	Alternatives écartées	Raison
ADR-09	Stockage d'objets compatible S3	Fichiers en base ou sur disque local	Volumétrie, sauvegardes, installation sur site

14. Décisions validées

Sujet	Décision	Prise en compte
Application mobile	Application Android développée en priorité.	ADR-11 ; section 6.
Authentification	Option la plus recommandée, choisie par l'architecte : Keycloak comme fournisseur d'identité.	ADR-08 ; sections 4.1, 10, 11. Keycloak est un composant d'infrastructure (comme Nginx ou PostgreSQL) : l'architecture reste client-serveur, sans microservices métier.
Code source	GitHub.	Dépôt GitHub, intégration continue GitHub Actions (section 4.3).
Portail et boutique	Option la plus adaptée, choisie par l'architecte : rendu côté serveur pour la boutique et les pages publiques, rendu navigateur pour le portail client connecté.	ADR-10 ; sections 3 et 6.
Reste du document	Validé sans modification.	—